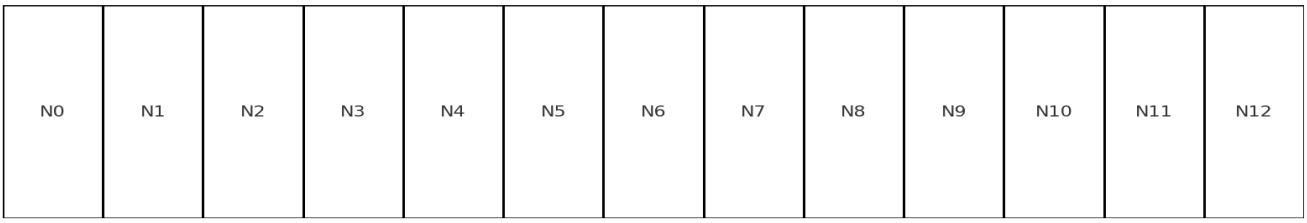
Mitutoyo 52-Bit Data Stream: Complete Guide (with Worked Example)

52-bit Frame Breakdown

Mitutoyo 52-bit Frame (13 nibbles, 4 bits each) — LSB nibble N0 on the left

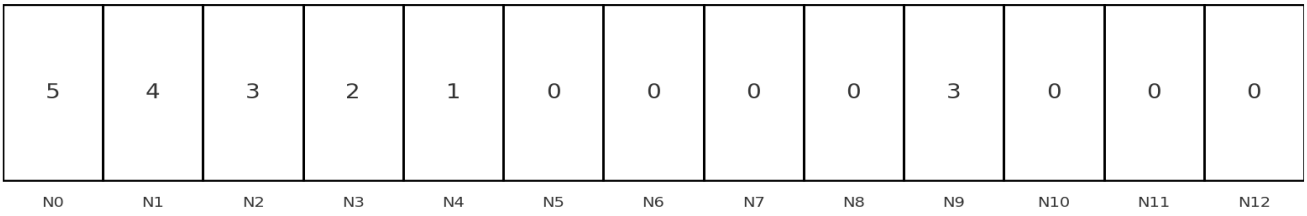


Wiring Connections

Caliper Pin	Signal	Arduino Pin (via level shifting)
1	GND	GND
2	DATA	D4
3	CLOCK	D2 (INT0)
4	REQ	D3 (optional)

Worked Example Decode

Worked Example — Raw Nibbles (LSB→MSB): 5 4 3 2 1 0 0 0 | 0 3 0 0 0



- Raw nibbles (LSB→MSB): N0..N12 = 5 4 3 2 1 0 0 0 | 0 3 0 0 0
- Digits (N0..N7) = 0..7 = [5,4,3,2,1,0,0,0] ⇒ integer value = 0012345 = 12,345 with dp not applied
- Sign (N8 = 0x0) ⇒ positive
- Decimal places (N9 = 0x3) ⇒ 3 decimal digits
- Units (N10 = 0x0) ⇒ millimeters (mm)
- Status (N11,N12 = 0x0,0x0) ⇒ no flags set
- Apply decimal: 12,345 / 10^3 = 12.345 ⇒ Final reading: 12.345 mm

Canonical Arduino Code

```
/*
  Mitutoyo / Digimatic 52-bit reader
  - Captures 52 bits LSB-first on CLK falling edge
  - Prints raw 13 nibbles and a best-effort decoded value
  - Works with SPC Digimatic calipers, micrometers, indicators
*/

#define CLK_PIN   2   // INT0
#define DATA_PIN  4
#define REQ_PIN   3

const bool USE_REQ = true;
const unsigned long REQ_LOW_MS  = 120;
const unsigned long FRAME_TIMEOUT_MS = 200;

volatile uint64_t frameBits = 0;
volatile uint8_t  bitCount  = 0;
volatile bool     frameReady = false;

void isrClockFalling() {
  uint8_t bit = digitalRead(DATA_PIN) ? 1 : 0;
  if (bitCount < 64) {
    frameBits |= (uint64_t)bit << bitCount;
    bitCount++;
    if (bitCount == 52) {
      frameReady = true;
      detachInterrupt(digitalPinToInterrupt(CLK_PIN));
    }
  }
}

void startCapture() {
  noInterrupts();
  frameBits = 0;
  bitCount  = 0;
  frameReady = false;
  interrupts();
  attachInterrupt(digitalPinToInterrupt(CLK_PIN), isrClockFalling, FALLING);
}

void sendRequestPulseIfEnabled() {
  if (!USE_REQ) return;
  digitalWrite(REQ_PIN, LOW);
  delay(REQ_LOW_MS);
  digitalWrite(REQ_PIN, HIGH);
}

void setup() {
  Serial.begin(115200);
  pinMode(CLK_PIN, INPUT_PULLUP);
  pinMode(DATA_PIN, INPUT_PULLUP);
  if (USE_REQ) {
    pinMode(REQ_PIN, OUTPUT);
    digitalWrite(REQ_PIN, HIGH);
  }
  Serial.println("Mitutoyo 52-bit reader ready.");
}

bool captureOneFrame(uint64_t &outBits) {
  startCapture();
  sendRequestPulseIfEnabled();
  unsigned long t0 = millis();
  while (!frameReady && (millis() - t0) < FRAME_TIMEOUT_MS) {}
  detachInterrupt(digitalPinToInterrupt(CLK_PIN));
  if (!frameReady) return false;
  noInterrupts();
```

```
  outBits = frameBits;
  interrupts();
  return true;
}

void printDecoded(uint64_t bits) {
  uint8_t nib[13];
  for (int i=0;i<13;i++) nib[i] = (bits >> (i*4)) & 0xF;

  long value = 0, place = 1;
  for (int i=0;i<8;i++){ value += (nib[i] % 10) * place; place *= 10; }

  bool neg = (nib[8] & 0x8) != 0;
  int dp   = nib[9] & 0x7;
  bool inch= (nib[10] & 0x1) != 0;

  long scale = 1; for(int i=0;i<dp;i++) scale *= 10;
  long ip = value / scale, fp = value % scale;

  Serial.print("Value: ");
  if (neg) Serial.print('-');
  Serial.print(ip);
  if (dp>0) { Serial.print('.'); if (fp < scale/10) for(int z=1; z<dp; z++) if (fp < (scale/(10*z))) Ser
  Serial.println(inch ? " in" : " mm");
}

void loop() {
  uint64_t bits;
  if (captureOneFrame(bits)) {
    Serial.print("Raw nibbles (LSB->MSB): ");
    for (int i = 0; i < 13; i++) { Serial.print((bits >> (i*4)) & 0xF, HEX); Serial.print(' '); }
    Serial.println();
    printDecoded(bits);
  }
  delay(50);
}
```

Use the raw-nibble printout to verify your device’s mapping. If a model varies, adjust the sign/decimal/unit interpretation in the decode function and note any status bits according to your instrument’s documentation.